

Frameworks

El siguiente código usa Flask, un framework web de Python.

Python

```
from flask import Flask, request, jsonify

app = Flask(__name__)

def validar_usuario(datos):
    return "nombre" in datos and "email" in datos

def guardar_en_base(datos):
    print(f"Guardando: {datos}")
    return 42

@app.route("/usuarios", methods=["POST"])
def crear_usuario():
    datos = request.get_json()
    if not validar_usuario(datos):
        return jsonify({"error": "datos incompletos"}), 400
    id_nuevo = guardar_en_base(datos)
    return jsonify({"id": id_nuevo}), 201

if __name__ == "__main__":
    app.run(debug=True)
```

- [☒ / F] `crear_usuario` es un hotspot del framework: es el punto donde el desarrollador introduce la lógica específica de la aplicación.
- [☒ / F] El mecanismo de ruteo de Flask (recibir la request HTTP, parsear la URL, despachar al handler correcto) es un frozen spot: el desarrollador no lo modifica ni lo reimplementa.
- [V / ☒] El desarrollador decide cuándo se llama a `crear_usuario`: lo invoca explícitamente cuando llega una request HTTP.
- [☒ / F] Flask decide cuándo llamar a `crear_usuario` en respuesta a una request HTTP entrante.
- [V / ☒] `validar_usuario` y `guardar_en_base` son frozen spots

Seguridad en lenguajes de programación

El siguiente código:

Python

```
def calcular_promedio(valores):  
    assert isinstance(valores, list), "valores debe ser una  
    lista"  
    assert len(valores) > 0, "la lista no puede estar vacía"  
    assert all(isinstance(v, (int, float)) for v in valores), \  
        "todos los valores deben ser numéricos"  
    return sum(valores) / len(valores)
```

- [V / **F**] Se trata de un ejemplo de programación ofensiva.
- [**V** / F] Se trata de un ejemplo de defensa por contrato: valida las precondiciones antes de modificar el estado.
- [**V** / F] Los asserts están ejerciendo una función equivalente a guardas.
- [V / **F**] Los asserts están ejerciendo una función equivalente a mecanismos de manejo de excepciones.

Scripting

En el siguiente código:

Python

```
1. def procesar_archivo(path):  
2.     resultado = {}  
3.     with open(path) as f:  
4.         for linea in f:  
5.             partes = linea.strip().split(",")  
6.             clave = partes[0]  
7.             valor = partes[1]  
8.             resultado[clave] = valor  
9.     return resultado  
10.  
11. datos = procesar_archivo("config.txt")  
12. datos["timeout"] = 30  
13. datos["reintentos"] = "3"  
14.  
15. for clave, valor in datos.items():  
16.     print(f"{clave} = {valor} (tipo: {type(valor).__name__})")
```

Para cada una de las siguientes propiedades, consigne el número de una línea del programa en la que se encuentre la propiedad, o ninguna si la propiedad no se encuentra en este fragmento de código:

Propiedad	Línea
Seguridad de tipos: los tipos de datos de las variables son suficientemente específicos como para evitar comportamientos inesperados.	
Las variables no requieren asignación explícita de tipo.	1
Buena comunicación con el sistema operativo: invoca directamente una herramienta del sistema desde el script.	3
Uso de expresiones regulares sofisticadas (cuantificadores, abstracciones sobre conjuntos de caracteres, agrupaciones, negaciones, etc.).	
Reglas de alcance explícitas.	
Construcciones lingüísticas de alto nivel propias del dominio de aplicación.	

Programación lógica

Dada la siguiente base de conocimiento:

None

```

especialista(ana, traumatologia).
especialista(elena, traumatologia).
especialista(clara, otorrinolaringologia).
especialista(esteban, traumatologia).
deriva(clara, ana).
deriva(clara, elena).
deriva(esteban, ana).

departamento(X, Y) :- especialista(X, A), especialista(Y, A).
jefe(X, Y) :- deriva(X, Y), departamento(X, Y).
equipo(X, Y) :- departamento(X,Y), not(jefe(X, Y)).

```

Le preguntamos al intérprete de Prolog si `equipo(elena, ana)` es True. Para cada uno de los siguientes subobjetivos, indique si el algoritmo de backtracking necesita que sea True para demostrar `equipo(elena, ana)`.

<code>especialista(elena,traumatologia).</code>	True False No se comprueba
<code>departamento(elena,ana).</code>	True False No se comprueba
<code>departamento(ana,elena).</code>	True False No se comprueba
<code>deriva(elena,ana).</code>	True False No se comprueba

Concurrencia

None

```
import scala.concurrent.{Future, Await}
import scala.concurrent.ExecutionContext.Implicits.global
import scala.concurrent.duration._

var contador = 0

// (A)
val t = new Thread(() => {
  Thread.sleep(100)
  contador += 1
})
t.start()

// (B)
val f1 = Future { Thread.sleep(300); "resultado 1" }
val f2 = Future { Thread.sleep(200); "resultado 2" }

// (C)
val combinado = Future.sequence(List(f1, f2))

// (D)
t.join()

// (E)
val resultados = Await.result(combinado, 5.seconds)
println(s"Contador: $contador, Resultados: $resultados")
```

Para cada fragmento identificado con una letra, indicá qué concepto o conceptos de concurrencia se aplican y por qué. Puede aplicar más de uno.

	Exclusión mútua	Sincronización	Manejo explícito de procesos	Ejecución concurrente
A			x	
B				x
C		x		x
D		x		
E		x		